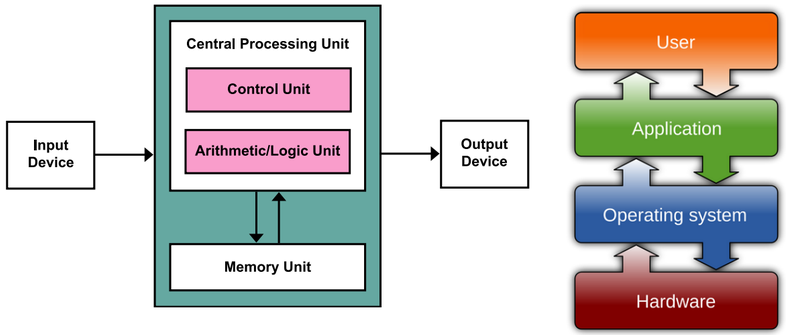
MoonCluster, a Physically Sandboxed Computer Architecture

Nicholas Belotserkovskiy, Joseph Lin, Alec Tippie, Niko Baletin

Introduction

MoonCluster is an alternative research personal computer architecture designed for reliably and securely sandboxing userspace processes, allowing the user to safely run programs without being infected by root-level malware.



Why It matters:

Modern systems can never provide complete security. Even the most advanced security-focused operating systems remain theoretically susceptible to sandbox escapes, which can have serious consequences for a system. In short, running any program carries inherent risk, and malware can potentially damage or even destroy any computer. MoonCluster offers a potential alternative.

What is an OS?

An operating system is the core software that manages a computer's hardware and provides the foundation on which applications run. It handles essential tasks like managing memory, coordinating processes, controlling input and output, and enforcing security so programs can run safely and efficiently. In short, the operating system acts as the intermediary between users, applications, and the hardware, making modern computing possible.

What we accomplished:

This semester, the team built a simulation of the user module and guard module and implemented basic Unix-like file system calls, including fopen, fclose, fwrite, fread, fseek, ftell, feof, and rewind. We tested by using our own implementation of the `cat` unix utility, which relied entirely on our custom system call functions.

How MoonCluster works

Unlike normal operating systems, MoonCluster physically sandboxes the user process to its own hardware, and forces the process to communicate with a file server to access files. Even if the userspace process is infected with malware, the device will never effect the rest of the system, and therefore will never access files outside of its permissions. If it were infected with malware, a factory reset is not necessary, as simply powering off the user module and rebooting would reset it to a stable state.